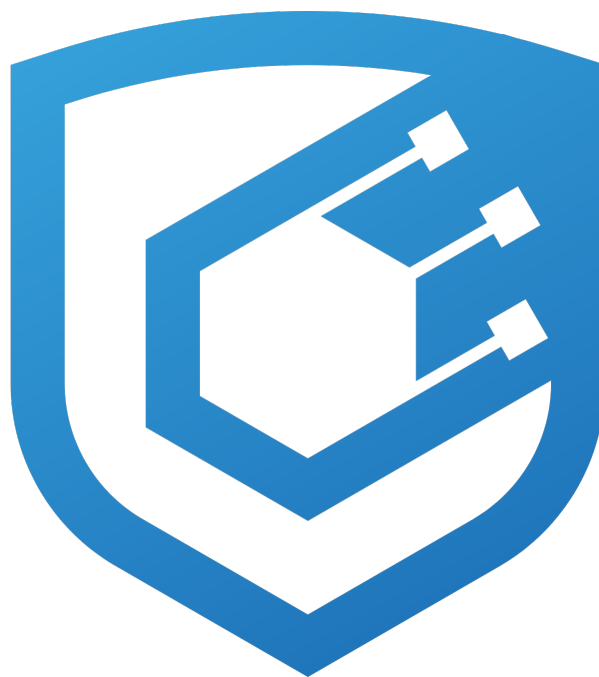




Securitize EVM Async Vault Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[rvierdiiev](#)

[InfiniteSec](#)

August 5, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	2
6.1	Centralization Risks	3
6.2	Protocol Invariants	3
7	Findings	6
7.1	High Risk	6
7.1.1	Nominal redemption accounting can make rebasing DS Token generations unfulfillable	6
7.2	Medium Risk	16
7.2.1	Double rounding in partial redemption allows reserve extraction without equivalent DS Token burn	16
7.3	Low Risk	18
7.3.1	AsyncFundVault::requestDeposit rejects documented NonWithdrawable submissions . . .	18
7.3.2	AsyncFundVault::requestDeposit permits dust grieving of controller capacity	18
7.3.3	AsyncFundVault::_executeRedeemClaim strands redemption rounding dust	18
7.3.4	AsyncFundVault::deposit strands zero-share fulfilled deposits	19
7.3.5	AsyncFundVault::fulfillRedemptions burns zero-value redemptions	19
7.3.6	AsyncFundVault::withdraw rejects zero-fill redemption claims	20
7.3.7	ERC 7540 and ERC 7575 conformance gaps break standard discovery and claim availability views	20
7.3.8	Aggregated deposit claims can exceed DS Token compliance limits	21
7.3.9	AsyncFundVault::totalAssets reports a mutable liquidity reserve rather than total managed assets	21
7.3.10	Zero live NAV bypasses settlement price validation	22
7.3.11	Cancellation functions do not support an alternative recipient	22
7.4	Informational	23
7.4.1	AsyncFundVault::redeem permits vault-self claim receivers	23
7.4.2	AsyncFundVault::initialize accepts non-contract NAV providers	23
7.4.3	Self reassignment temporarily hides claims from aggregate indexes until administrator repair	23
7.4.4	Partial redemption claims require administrative recovery if the controller loses DS Token eligibility	25
7.5	Gas Optimization	27
7.5.1	Cache request balances that are checked before an additive update	27
7.5.2	Use the already-snapshotted list length while compacting claim lists	27
7.5.3	Pack the tolerance with the cached token decimal fields	28
7.5.4	Cache each reused generation mapping entry as a storage reference	28

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Securitize Async Ramp is an ERC-7540 asynchronous vault that batches investor subscriptions and redemptions for DS Tokens. Investors submit deposit or redemption requests into an active generation; a settler later fulfills the generation with a fixed NAV price, and investors claim all fulfilled generations for their controller in one transaction.

The deposit path escrows liquidity tokens before settlement and issues DS Tokens at claim time, so DS Token compliance is checked when shares are actually minted. The redemption path escrows DS Tokens, calculates a generation-level fill rate from available liquidity and NAV, burns the fulfilled portion in bulk, and returns each investor's computed pro-rata unfulfilled share amount when the investor claims; rounding residuals are described in the findings report.

The vault tracks reserve liquidity internally rather than relying on token balances. Manager actions can inject or withdraw reserve liquidity, while withdrawal headroom must preserve liquidity already committed to fulfilled redemptions and cancellable active deposits. Operators can act for controllers only after controller approval, and the default admin retains upgrade, pause, reassignment, and configuration powers.

5 Audit Scope

The audit scope was limited to:

```
contracts/base/AsyncFundVaultAdmin.sol
contracts/base/AsyncFundVaultStorage.sol
contracts/AsyncFundVault.sol
```

6 Executive Summary

Over the course of 5 days, the Cyfrin team conducted an audit on the [EVM Async Vault](#) code provided by [Securitize](#). The findings consist of 1 High, 1 Medium & 11 Low severity issues with the remainder being informational and gas optimizations.

6.1 Centralization Risks

- The default admin can upgrade the vault implementation, pause or unpause investor flows, change NAV tolerance, and reassign fulfilled claims; users rely on this role being operated correctly and without a malicious upgrade.
- The manager controls reserve operations and subscription or redemption availability through `AsyncFundVaultAdmin::injectLiquidity`, `withdrawReserve`, `setSubscriptionsEnabled`, and `setRedemptionsEnabled`; incorrect or delayed operation can affect when investors can enter, exit, or receive liquidity.
- The settler controls generation lifecycle and fulfillment through `AsyncFundVault::openDepositGeneration`, `openRedemptionGeneration`, `fulfillDeposits`, and `fulfillRedemptions`; investors depend on the settler to open windows, strike NAV, and advance fulfilled generations.
- DS Token issuance and burning depend on the vault proxy holding `ROLE_ISSUER` in the external DS Protocol TrustService, and direct TokenIssuer issuance is outside the documented operational path.
- The transfer agent is an off-chain trust dependency for NAV striking and DS Token compliance administration, including blocked-wallet management through the external DS Token stack.
- Deployment operations for the DS Token and NAV provider load signer keys from environment-backed configuration; production deployments should treat those signers as privileged operational keys and protect them accordingly.
- Administrative claim reassignment can append fulfilled generation IDs to the destination controller without enforcing `MAX_OUTSTANDING_GENERATIONS`, so an administrator can grow a controller's claim list beyond the cap that normal request paths enforce.
- Same-address claim reassignment can remove a fulfilled generation from the controller index while retaining its mapping balance, and reserve withdrawal to the vault itself can decrement reserve accounting without moving custody.
- Administrative reassignment and reserve-management paths can recover or move value only if the privileged operator chooses the correct destination, token behavior, and timing; these powers are trust assumptions rather than permissionless user controls.

6.2 Protocol Invariants

- Reserve solvency: `reserveBalance` is always at least `totalClaimableRedemptionLiquidity` plus the pending deposits of the current Active deposit generation, enforced by the subtraction in `AsyncFundVaultAdmin::withdrawReserve` and `AsyncFundVault::fulfillRedemptions`.
- Share conservation: for each redemption generation, the sum of unfulfilled DS Token returns across all claimants never exceeds `totalPendingShares` minus the amount burned in bulk at fulfillment.
- Lifecycle monotonicity: a generation ID never returns to an earlier status, at most one generation per flow is Active, and Fulfilled is terminal.
- Index consistency: a generation ID is present in a controller's generation list exactly while that controller holds a nonzero pending balance in the corresponding generation mapping.
- Outstanding generation cap: a controller's generation lists remain at or below `MAX_OUTSTANDING_GENERATIONS`, which is 30.
- NAV bound: the `navPrice` recorded on a fulfilled generation lies within the configured tolerance of the live NAV price, except when the NAV provider returns zero and the check is skipped.

Summary

Project Name	EVM Async Vault
Repository	bc-async-ramp-sc
Commit	89c2a850e73a...
Fix Commit	6aaeb7aceb8a...
Audit Timeline	July 22nd - July 28th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	1
Medium Risk	1
Low Risk	11
Informational	4
Gas Optimizations	4
Total Issues	21

Summary of Findings

[H-1] Nominal redemption accounting can make rebasing DS Token generations unfulfillable	Acknowledged
[M-1] Double rounding in partial redemption allows reserve extraction without equivalent DS Token burn	Resolved
[L-01] AsyncFundVault::requestDeposit rejects documented NonWithdrawable submissions	Resolved
[L-02] AsyncFundVault::requestDeposit permits dust grieving of controller capacity	Resolved
[L-03] AsyncFundVault::_executeRedeemClaim strands redemption rounding dust	Acknowledged
[L-04] AsyncFundVault::deposit strands zero-share fulfilled deposits	Resolved
[L-05] AsyncFundVault::fulfillRedemptions burns zero-value redemptions	Resolved
[L-06] AsyncFundVault::withdraw rejects zero-fill redemption claims	Resolved
[L-07] ERC 7540 and ERC 7575 conformance gaps break standard discovery and claim availability views	Resolved
[L-08] Aggregated deposit claims can exceed DS Token compliance limits	Acknowledged
[L-09] AsyncFundVault::totalAssets reports a mutable liquidity reserve rather than total managed assets	Resolved
[L-10] Zero live NAV bypasses settlement price validation	Resolved

[L-11] Cancellation functions do not support an alternative recipient	Resolved
[I-1] AsyncFundVault::redeem permits vault-self claim receivers	Resolved
[I-2] AsyncFundVault::initialize accepts non-contract NAV providers	Acknowledged
[I-3] Self reassignment temporarily hides claims from aggregate indexes until administrator repair	Resolved
[I-4] Partial redemption claims require administrative recovery if the controller loses DS Token eligibility	Resolved
[G-1] Cache request balances that are checked before an additive update	Resolved
[G-2] Use the already-snapshotted list length while compacting claim lists	Acknowledged
[G-3] Pack the tolerance with the cached token decimal fields	Resolved
[G-4] Cache each reused generation mapping entry as a storage reference	Acknowledged

7 Findings

7.1 High Risk

7.1.1 Nominal redemption accounting can make rebasing DS Token generations unfulfillable

Description: AsyncFundVault::requestRedeem records requests in nominal DS Token units:

```
$.pendingRedeemShares[genId][controller] += shares;
$.redeemGenerations[genId].totalPendingShares += shares;

IERC20(address($.dsToken)).safeTransferFrom(
    owner,
    address(this),
    shares
);
```

The DS Token internally converts each nominal transfer into rebasing shares. Later, fulfillRedemptions aggregates the stored nominal requests and performs a single nominal burn:

```
uint256 burnAmount =
    (totalShares * fulfillmentRate) / WAD;

$.dsToken.burn(
    address(this),
    burnAmount,
    "AsyncFundVault: redemption"
);
```

Because the vault never records the internal shares actually received, the aggregate burn may require more internal shares than the vault escrowed. This can occur through two trigger paths:

1. **Multiplier change:** If the rebasing multiplier changes between request and fulfillment, converting the stored nominal amount during burn produces a different internal-share amount than the original transfers.
2. **Independent rounding:** Even with a constant multiplier, individually converting each request and later converting their nominal aggregate can produce different results. For example, with a multiplier of 10, two requests of 19 tokens each escrow $\text{floor}(19 / 10) + \text{floor}(19 / 10) = 1 + 1 = 2$ internal shares but burning their aggregate requires: $\text{floor}((19 + 19) / 10) = \text{floor}(38 / 10) = 3$ internal shares. The vault therefore escrows 2 internal shares but attempts to burn 3.

In either case, the vault's nominal generation accounting does not reconcile with its actual internal-share balance.

Impact: fulfillRedemptions can revert for the entire generation because the vault lacks enough internal shares for the aggregate burn. Once the generation is closed, users cannot cancel, and because fulfillment cannot succeed, they cannot claim through the normal flow. The generation remains blocked until an external action restores sufficient balance, such as a favorable rebase, administrative recovery, or contract upgrade.

Proof of Concept:

1. **Multiplier change case:** Add the following Foundry test file and run:

```
forge test --match-test test_PoC_RebasingEscrowUnderflowBlocksFulfillment -vvv
```

```
pragma solidity ^0.8.22;

import {Test} from "forge-std/Test.sol";
import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";

import {AsyncFundVault} from "../contracts/AsyncFundVault.sol";
import {AsyncFundVaultStorage} from "../contracts/base/AsyncFundVaultStorage.sol";
import {IAsyncFundVaultErrors} from "../contracts/interfaces/IAsyncFundVaultErrors.sol";
```

```

contract ConfigurableERC20 {
    string public name;
    string public symbol;
    uint8 private immutable tokenDecimals;
    uint256 public totalSupply;

    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;

    event Transfer(address indexed from, address indexed to, uint256 value);
    event Approval(address indexed owner, address indexed spender, uint256 value);

    constructor(string memory name_, string memory symbol_, uint8 decimals_) {
        name = name_;
        symbol = symbol_;
        tokenDecimals = decimals_;
    }

    function decimals() external view returns (uint8) {
        return tokenDecimals;
    }

    function approve(address spender, uint256 value) external returns (bool) {
        allowance[msg.sender][spender] = value;
        emit Approval(msg.sender, spender, value);
        return true;
    }

    function transfer(address to, uint256 value) external returns (bool) {
        _transfer(msg.sender, to, value);
        return true;
    }

    function transferFrom(address from, address to, uint256 value) external returns (bool) {
        uint256 approved = allowance[from][msg.sender];
        if (approved != type(uint256).max) {
            allowance[from][msg.sender] = approved - value;
            emit Approval(from, msg.sender, allowance[from][msg.sender]);
        }

        _transfer(from, to, value);
        return true;
    }

    function mint(address to, uint256 value) public {
        totalSupply += value;
        balanceOf[to] += value;
        emit Transfer(address(0), to, value);
    }

    function _transfer(address from, address to, uint256 value) internal {
        require(to != address(0), "TRANSFER_TO_ZERO");
        balanceOf[from] -= value;
        balanceOf[to] += value;
        emit Transfer(from, to, value);
    }
}

contract RebaseLikeDSToken is ConfigurableERC20 {
    constructor(uint8 decimals_) ConfigurableERC20("Rebase DS", "rDS", decimals_) {}

    function issueTokens(address to, uint256 value) external returns (bool) {
        mint(to, value);
    }
}

```



```

        return true;
    }

    function burn(address who, uint256 value, string calldata) external {
        require(balanceOf[who] >= value, "BURN_EXCEEDS_BALANCE");
        balanceOf[who] -= value;
        totalSupply -= value;
        emit Transfer(who, address(0), value);
    }

    function preTransferCheck(address, address, uint256) external pure returns (uint256, string memory)
    ↪ {
        return (0, "");
    }

    function setRebasedBalance(address account, uint256 newBalance) external {
        uint256 oldBalance = balanceOf[account];
        if (newBalance > oldBalance) {
            totalSupply += newBalance - oldBalance;
        } else {
            totalSupply -= oldBalance - newBalance;
        }
        balanceOf[account] = newBalance;
    }
}

contract AssetDecimalNavProvider {
    uint256 public rate;

    constructor(uint256 rate_) {
        rate = rate_;
    }

    function setRate(uint256 rate_) external {
        rate = rate_;
    }
}

contract AsyncFundVaultPoC_Test is Test {
    address private admin = makeAddr("admin");
    address private settler = makeAddr("settler");
    address private manager = makeAddr("manager");
    address private alice = makeAddr("alice");
    address private bob = makeAddr("bob");

    function test_PoC_RebasingEscrowUnderflowBlocksFulfillment() public {
        (AsyncFundVault vault, RebaseLikeDSToken dsToken, ConfigurableERC20 liquidityToken) =
            _deployVault(6, 6, 100e6);

        uint256 aliceRedemptionShares = 2e6;
        uint256 bobRedemptionShares = 1e6;
        uint256 totalRedemptionShares = aliceRedemptionShares + bobRedemptionShares;
        _injectLiquidity(vault, liquidityToken, 300e6);

        dsToken.mint(alice, aliceRedemptionShares);
        vm.startPrank(alice);
        dsToken.approve(address(vault), aliceRedemptionShares);
        vault.requestRedeem(aliceRedemptionShares, alice, alice);
        vm.stopPrank();

        dsToken.mint(bob, bobRedemptionShares);
        vm.startPrank(bob);
        dsToken.approve(address(vault), bobRedemptionShares);
    }
}

```

```

    vault.requestRedeem(bobRedemptionShares, bob, bob);
    vm.stopPrank();

    assertEq(vault.pendingRedeemRequest(0, alice), aliceRedemptionShares);
    assertEq(vault.pendingRedeemRequest(0, bob), bobRedemptionShares);
    assertEq(dsToken.balanceOf(address(vault)), totalRedemptionShares);

    vm.prank(settler);
    vault.closeRedemptionGeneration(0);

    dsToken.setRebasedBalance(address(vault), totalRedemptionShares / 2);

    assertEq(vault.pendingRedeemRequest(0, alice), aliceRedemptionShares);
    assertEq(vault.pendingRedeemRequest(0, bob), bobRedemptionShares);
    assertEq(dsToken.balanceOf(address(vault)), totalRedemptionShares / 2);

    vm.prank(settler);
    vm.expectRevert(bytes("BURN_EXCEEDS_BALANCE"));
    vault.fulfillRedemptions(0, 100e18);

    AsyncFundVaultStorage.RedemptionGenerationData memory generation =
    ↪ vault.getRedemptionGeneration(0);
    assertEq(uint8(generation.status), uint8(AsyncFundVaultStorage.GenerationStatus.Closed));
    assertEq(vault.claimableRedeemRequest(0, alice), 0);
    assertEq(vault.claimableRedeemRequest(0, bob), 0);
    assertEq(vault.totalClaimableRedemption(alice), 0);
    assertEq(vault.totalClaimableRedemption(bob), 0);

    vm.prank(alice);
    vm.expectRevert(
        abi.encodeWithSelector(IAsyncFundVaultErrors.CancellationNotAllowed.selector, uint256(0))
    );
    vault.cancelRedeemRequest(0, alice);

    vm.prank(bob);
    vm.expectRevert(
        abi.encodeWithSelector(IAsyncFundVaultErrors.CancellationNotAllowed.selector, uint256(0))
    );
    vault.cancelRedeemRequest(0, bob);

    vm.prank(alice);
    vm.expectRevert(
        abi.encodeWithSelector(IAsyncFundVaultErrors.NoClaimableRedemption.selector, alice)
    );
    vault.redeem(aliceRedemptionShares, alice, alice);

    vm.prank(bob);
    vm.expectRevert(
        abi.encodeWithSelector(IAsyncFundVaultErrors.NoClaimableRedemption.selector, bob)
    );
    vault.redeem(bobRedemptionShares, bob, bob);
}

function _deployVault(uint8 dsDecimals, uint8 liquidityDecimals, uint256 navRate)
    private
    returns (AsyncFundVault vault, RebaseLikeDSToken dsToken, ConfigurableERC20 liquidityToken)
{
    dsToken = new RebaseLikeDSToken(dsDecimals);
    liquidityToken = new ConfigurableERC20("Liquidity", "LIQ", liquidityDecimals);
    AssetDecimalNavProvider navProvider = new AssetDecimalNavProvider(navRate);

    vm.startPrank(admin);
    AsyncFundVault impl = new AsyncFundVault();

```

```

        bytes memory initData = abi.encodeCall(
            AsyncFundVault.initialize,
            (address(dsToken), address(liquidityToken), address(navProvider))
        );
        vault = AsyncFundVault(address(new ERC1967Proxy(address(impl), initData)));
        vault.grantRole(vault.SETTLER_ROLE(), settler);
        vault.grantRole(vault.MANAGER_ROLE(), manager);
        vm.stopPrank();

        vm.startPrank(settler);
        vault.openDepositGeneration();
        vault.openRedemptionGeneration();
        vm.stopPrank();
    }

    function _injectLiquidity(AsyncFundVault vault, ConfigurableERC20 liquidityToken, uint256 amount)
    private {
        liquidityToken.mint(manager, amount);
        vm.startPrank(manager);
        liquidityToken.approve(address(vault), amount);
        vault.injectLiquidity(amount);
        vm.stopPrank();
    }
}

```

2. Independent rounding case: Add the following Foundry test file and run:

```
forge test --match-test test_PoC_AggregateBurnExceedsRoundedEscrowShares -vvv
```

```

pragma solidity ^0.8.22;

import {Test} from "forge-std/Test.sol";
import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";

import {AsyncFundVault} from "../contracts/AsyncFundVault.sol";
import {AsyncFundVaultStorage} from "../contracts/base/AsyncFundVaultStorage.sol";
import {IAsyncFundVaultErrors} from "../contracts/interfaces/IAsyncFundVaultErrors.sol";

contract ConfigurableERC20 {
    string public name;
    string public symbol;
    uint8 private immutable tokenDecimals;
    uint256 public totalSupply;

    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;

    event Transfer(address indexed from, address indexed to, uint256 value);
    event Approval(address indexed owner, address indexed spender, uint256 value);

    constructor(string memory name_, string memory symbol_, uint8 decimals_) {
        name = name_;
        symbol = symbol_;
        tokenDecimals = decimals_;
    }

    function decimals() external view returns (uint8) {
        return tokenDecimals;
    }

    function approve(address spender, uint256 value) external returns (bool) {
        allowance[msg.sender][spender] = value;
        emit Approval(msg.sender, spender, value);
    }
}

```

```

        return true;
    }

    function transfer(address to, uint256 value) external returns (bool) {
        _transfer(msg.sender, to, value);
        return true;
    }

    function transferFrom(address from, address to, uint256 value) external returns (bool) {
        uint256 approved = allowance[from][msg.sender];
        if (approved != type(uint256).max) {
            allowance[from][msg.sender] = approved - value;
            emit Approval(from, msg.sender, allowance[from][msg.sender]);
        }

        _transfer(from, to, value);
        return true;
    }

    function mint(address to, uint256 value) public {
        totalSupply += value;
        balanceOf[to] += value;
        emit Transfer(address(0), to, value);
    }

    function _transfer(address from, address to, uint256 value) internal {
        require(to != address(0), "TRANSFER_TO_ZERO");
        balanceOf[from] -= value;
        balanceOf[to] += value;
        emit Transfer(from, to, value);
    }
}

contract RoundingRebaseDSToken {
    uint256 private constant WAD = 1e18;

    string public name = "Rounding DS";
    string public symbol = "rDS";
    uint8 private immutable tokenDecimals;
    uint256 public immutable multiplier;
    uint256 private totalShareSupply;

    mapping(address => uint256) private shareBalance;
    mapping(address => mapping(address => uint256)) public allowance;

    event Transfer(address indexed from, address indexed to, uint256 value);
    event Approval(address indexed owner, address indexed spender, uint256 value);

    constructor(uint8 decimals_, uint256 multiplier_) {
        tokenDecimals = decimals_;
        multiplier = multiplier_;
    }

    function decimals() external view returns (uint8) {
        return tokenDecimals;
    }

    function totalSupply() external view returns (uint256) {
        return _sharesToTokens(totalShareSupply);
    }

    function balanceOf(address account) external view returns (uint256) {
        return _sharesToTokens(shareBalance[account]);
    }
}

```

```

}

function rawSharesOf(address account) external view returns (uint256) {
    return shareBalance[account];
}

function approve(address spender, uint256 value) external returns (bool) {
    allowance[msg.sender][spender] = value;
    emit Approval(msg.sender, spender, value);
    return true;
}

function transfer(address to, uint256 value) external returns (bool) {
    _transferTokens(msg.sender, to, value);
    return true;
}

function transferFrom(address from, address to, uint256 value) external returns (bool) {
    uint256 approved = allowance[from][msg.sender];
    if (approved != type(uint256).max) {
        allowance[from][msg.sender] = approved - value;
        emit Approval(from, msg.sender, allowance[from][msg.sender]);
    }

    _transferTokens(from, to, value);
    return true;
}

function mint(address to, uint256 value) public {
    uint256 shares = _tokensToShares(value);
    shareBalance[to] += shares;
    totalShareSupply += shares;
    emit Transfer(address(0), to, value);
}

function issueTokens(address to, uint256 value) external returns (bool) {
    mint(to, value);
    return true;
}

function burn(address who, uint256 value, string calldata) external {
    uint256 shares = _tokensToShares(value);
    require(shares <= shareBalance[who], "Not enough balance");
    shareBalance[who] -= shares;
    totalShareSupply -= shares;
    emit Transfer(who, address(0), value);
}

function preTransferCheck(address, address, uint256) external pure returns (uint256, string memory)
↳ {
    return (0, "");
}

function _transferTokens(address from, address to, uint256 value) internal {
    require(to != address(0), "TRANSFER_TO_ZERO");
    uint256 shares = _tokensToShares(value);
    require(shares <= shareBalance[from], "Not enough balance");
    shareBalance[from] -= shares;
    shareBalance[to] += shares;
    emit Transfer(from, to, value);
}

function _tokensToShares(uint256 tokens) internal view returns (uint256) {

```

```

        uint256 shares = (tokens * WAD + multiplier / 2) / multiplier;
        if (tokens > 0) {
            require(shares > 0, "Shares amount too small");
        }
        return shares;
    }

    function _sharesToTokens(uint256 shares) internal view returns (uint256) {
        return (shares * multiplier + WAD / 2) / WAD;
    }
}

contract AssetDecimalNavProvider {
    uint256 public rate;

    constructor(uint256 rate_) {
        rate = rate_;
    }

    function setRate(uint256 rate_) external {
        rate = rate_;
    }
}

contract AsyncFundVaultPoC_Test is Test {
    address private admin = makeAddr("admin");
    address private settler = makeAddr("settler");
    address private manager = makeAddr("manager");
    address private alice = makeAddr("alice");
    address private bob = makeAddr("bob");

    function test_PoC_AggregateBurnExceedsRoundedEscrowShares() public {
        (AsyncFundVault vault, RoundingRebaseDSToken dsToken, ConfigurableERC20 liquidityToken) =
            _deployVaultWithRoundingToken(18, 18, 1e18, 10e18);

        uint256 aliceRedemptionShares = 14;
        uint256 bobRedemptionShares = 14;
        uint256 totalRedemptionShares = aliceRedemptionShares + bobRedemptionShares;
        _injectLiquidity(vault, liquidityToken, totalRedemptionShares);

        dsToken.mint(alice, aliceRedemptionShares);
        vm.startPrank(alice);
        dsToken.approve(address(vault), aliceRedemptionShares);
        vault.requestRedeem(aliceRedemptionShares, alice, alice);
        vm.stopPrank();

        dsToken.mint(bob, bobRedemptionShares);
        vm.startPrank(bob);
        dsToken.approve(address(vault), bobRedemptionShares);
        vault.requestRedeem(bobRedemptionShares, bob, bob);
        vm.stopPrank();

        AsyncFundVaultStorage.RedemptionGenerationData memory generation =
            ↪ vault.getRedemptionGeneration(0);
        assertEq(generation.totalPendingShares, totalRedemptionShares);
        assertEq(dsToken.rawSharesOf(address(vault)), 2);

        vm.prank(settler);
        vault.closeRedemptionGeneration(0);

        vm.prank(settler);
        vm.expectRevert(bytes("Not enough balance"));
        vault.fulfillRedemptions(0, 1e18);
    }
}

```

```

generation = vault.getRedemptionGeneration(0);
assertEq(uint8(generation.status), uint8(AsyncFundVaultStorage.GenerationStatus.Closed));
assertEq(generation.totalPendingShares, totalRedemptionShares);
assertEq(vault.claimableRedeemRequest(0, alice), 0);
assertEq(vault.claimableRedeemRequest(0, bob), 0);
assertEq(dsToken.rawSharesOf(address(vault)), 2);

vm.prank(alice);
vm.expectRevert(
    abi.encodeWithSelector(IAsyncFundVaultErrors.CancellationNotAllowed.selector, uint256(0))
);
vault.cancelRedeemRequest(0, alice);

vm.prank(bob);
vm.expectRevert(
    abi.encodeWithSelector(IAsyncFundVaultErrors.CancellationNotAllowed.selector, uint256(0))
);
vault.cancelRedeemRequest(0, bob);

vm.prank(alice);
vm.expectRevert(
    abi.encodeWithSelector(IAsyncFundVaultErrors.NoClaimableRedemption.selector, alice)
);
vault.redeem(aliceRedemptionShares, alice, alice);

vm.prank(bob);
vm.expectRevert(
    abi.encodeWithSelector(IAsyncFundVaultErrors.NoClaimableRedemption.selector, bob)
);
vault.redeem(bobRedemptionShares, bob, bob);
}

function _deployVaultWithRoundingToken(
    uint8 dsDecimals,
    uint8 liquidityDecimals,
    uint256 navRate,
    uint256 multiplier
)
private
returns (AsyncFundVault vault, RoundingRebaseDSToken dsToken, ConfigurableERC20 liquidityToken)
{
    dsToken = new RoundingRebaseDSToken(dsDecimals, multiplier);
    liquidityToken = new ConfigurableERC20("Liquidity", "LIQ", liquidityDecimals);
    AssetDecimalNavProvider navProvider = new AssetDecimalNavProvider(navRate);

    vm.startPrank(admin);
    AsyncFundVault impl = new AsyncFundVault();
    bytes memory initData = abi.encodeCall(
        AsyncFundVault.initialize,
        (address(dsToken), address(liquidityToken), address(navProvider))
    );
    vault = AsyncFundVault(address(new ERC1967Proxy(address(impl), initData)));
    vault.grantRole(vault.SETTLER_ROLE(), settler);
    vault.grantRole(vault.MANAGER_ROLE(), manager);
    vm.stopPrank();

    vm.startPrank(settler);
    vault.openDepositGeneration();
    vault.openRedemptionGeneration();
    vm.stopPrank();
}

```

```

function _injectLiquidity(AsyncFundVault vault, ConfigurableERC20 liquidityToken, uint256 amount)
↪ private {
    liquidityToken.mint(manager, amount);
    vm.startPrank(manager);
    liquidityToken.approve(address(vault), amount);
    vault.injectLiquidity(amount);
    vm.stopPrank();
}
}

```

Recommended Mitigation: Make redemption accounting consistently use internal rebasing shares. Record the internal shares actually received for each request and use that basis across fulfillment, cancellation, reassignment, claims, and views.

Securitize: Confirmed - we reproduced this against our own repo and the mechanism checks out: AsyncFundVault tracks deposits and redemptions purely in nominal DS Token units and never reconciles against the token's own internal share/rebasing accounting. Against a DS Token with an active or changing rebasing multiplier, fulfillRedemptions can indeed attempt to burn more internal shares than the vault actually holds, permanently stranding the generation.

For this version, we're addressing it as a documented constraint in commit [5cbf38b](#) rather than a code change: this vault is only supported against a DS Token whose rebasing multiplier is fixed at 1e18 (i.e. no active rebasing). We've added explicit warnings at the contract level, on initialize()'s dsToken param, and on the storage field itself, so this constraint is visible to anyone deploying or reviewing the contract, not just something known informally.

We're not treating this as "won't fix" - if a future fund needs this vault to work with an actively-rebasing DS Token, that will need a new version that tracks internal shares received/burned directly (per the recommended mitigation) rather than nominal amounts, since that's a real architectural change to the redemption accounting, not a patch. For now, every 7540 vault we deploy will be paired with a non-rebasing DS Token, so this is out of reach in practice.

7.2 Medium Risk

7.2.1 Double rounding in partial redemption allows reserve extraction without equivalent DS Token burn

Description: `AsyncFundVault::fulfillRedemptions` can commit more liquidity than the economic value of the DS Tokens it actually burns. A redeemer can consequently receive reserve assets while retaining part or all of the DS Tokens that should have funded that payment.

For a partially funded generation, the function first rounds the generation-wide fulfillment rate down, assigns all available liquidity to the generation, and then rounds the DS Token burn down a second time. The committed liquidity is not recomputed from the resulting integer burn amount.

```
// contracts/AsyncFundVault.sol
if (availableLiquidity >= totalRedemptionValue) {
    fulfillmentRate = WAD;
    totalLiquidityCommitted = totalRedemptionValue;
} else {
    fulfillmentRate = (availableLiquidity * WAD) / totalRedemptionValue;
    totalLiquidityCommitted = availableLiquidity;
}

uint256 burnAmount = (totalShares * fulfillmentRate) / WAD;
if (burnAmount > 0) {
    $.dsToken.burn(address(this), burnAmount, "AsyncFundVault: redemption");
}
```

This composes two floor operations as follows.

```
fulfillmentRate = floor(availableLiquidity * WAD / totalRedemptionValue)
burnAmount = floor(totalShares * fulfillmentRate / WAD)
totalLiquidityCommitted = availableLiquidity
```

The required solvency relationship is that `totalLiquidityCommitted` must not exceed the NAV value of `burnAmount`. The current formulas do not preserve that relationship. For example, consider a two-decimal DS Token with a NAV of 1,000,000 liquidity tokens. Each DS Token base unit is worth 10,000 liquidity tokens. If a redeemer requests three base units and the vault has 20,000 liquidity tokens available, `totalRedemptionValue` is 30,000 and the rounded fulfillment rate is 666666666666666666. Multiplying three by this rate and rounding down burns only one base unit, worth 10,000, even though the generation commits 20,000. On claim, the redeemer receives the full 20,000 and gets the other two base units back.

The more extreme boundary occurs when available liquidity is worth less than one DS Token base unit. The calculated `burnAmount` is then zero while `totalLiquidityCommitted` remains all available liquidity. The redeemer receives that liquidity and recovers the complete DS Token request, allowing the same tokens to be submitted again in a later generation. This does not permit an unbounded zero-burn withdrawal in a single generation. For an exactly linear NAV conversion and `totalShares < WAD`, the overcommit is strictly less than one DS Token base-unit value plus `totalRedemptionValue / WAD`; the ordinary asset-conversion floor adds only its base-unit residue. Repetition also requires the settler to progress through additional generations.

Impact: Partial fulfillment can release more liquidity than the NAV value of the DS Tokens burned while returning the unburned portion with its economic rights intact. The client confirmed that supported DS Token precision ranges from 0 to 18 decimals, normally six. At low precision, one base unit can represent a full economically valuable DS Token, and the zero-burn boundary returns that token for reuse in later generations after liquidity has been paid. Because the code imposes no NAV cap, repeated partial settlements can produce material reserve loss in a supported configuration, which supports Medium. Ordinary six-decimal configurations reduce the per-generation discrepancy to dust.

Recommended Mitigation: Finalize the integer burn amount first, then derive committed liquidity from that exact burn and leave any residual uncommitted. Enforce that committed liquidity never exceeds the NAV value removed from circulation, including zero-burn boundaries.

Securitize: Fixed in commit [cec2082](#).

Cyfrin: Verified.

7.3 Low Risk

7.3.1 AsyncFundVault::requestDeposit rejects documented NonWithdrawable submissions

Description: AsyncFundVault::requestDeposit accepts requests only when the current deposit generation is exactly GenerationStatus.Active; the same exact-status check appears in AsyncFundVault::requestRedeem. A generation in NonWithdrawable therefore rejects new subscription and redemption requests with NoActiveGeneration, although that lifecycle state is documented as accepting requests while disallowing cancellations (per docs/flows.md). See contracts/AsyncFundVault.sol:381-385 and contracts/AsyncFundVault.sol:579-582.

Impact: Investors can be excluded from the documented submission window during normal settlement operations and must wait for a later generation to submit either a subscription or redemption request.

Recommended Mitigation: In AsyncFundVault::requestDeposit and requestRedeem, accept both Active and NonWithdrawable generations if the documented lifecycle is intended. Keep the Active-only checks in cancelDepositRequest and cancelRedeemRequest, and account for newly accepted non-cancellable deposits when calculating reserved liquidity.

Securitize: Fixed in commit [b239f73](#).

Cyfrin: Verified.

7.3.2 AsyncFundVault::requestDeposit permits dust grieving of controller capacity

Description: AsyncFundVault::requestDeposit lets an authorized owner create a request for an arbitrary controller and adds the generation to that controller's bounded list before transferring the nominal asset amount. If a fulfilled request converts to zero shares, deposit and mint reject the claim before clearing that list entry, while cancelDepositRequest permits cancellation only for an Active generation. The zero-share condition is reachable when the DS token has materially fewer decimals than the liquidity token, or when the locked NAV is high enough that the smallest deposited asset amount rounds below one DS-token base unit. A third party can therefore create fulfilled, zero-share entries for another controller until the list reaches its capacity. See contracts/AsyncFundVault.sol:375-400, contracts/AsyncFundVault.sol:481-493, and contracts/AsyncFundVault.sol:532-559.

Impact: An unconsenting controller can be prevented from creating further deposit requests after dust entries consume its generation-list capacity. The resulting fulfilled entries cannot be cleared through the normal claim or cancellation paths.

Recommended Mitigation: Require controller consent when owner and controller differ, and add a fulfilled zero-share cleanup path that removes the generation entry and refunds its recorded assets. Preserve those refundable fulfilled deposits in reserve-withdrawal accounting until they are claimed or refunded, so a manager cannot withdraw the custody needed by the cleanup path. A settlement-safe minimum request amount can provide an additional guard, but must not replace the cleanup path.

Securitize: Fixed in commit [eca80d4](#).

Cyfrin: Verified.

7.3.3 AsyncFundVault::_executeRedeemClaim strands redemption rounding dust

Description: Redemption settlement locks a generation-level liquidity amount and bulk-burns fulfilled shares, but _executeRedeemClaim independently floors each controller's liquidity payout and unfulfilled-share return. It clears each controller's request and decrements the claimable-liquidity counter only by those rounded payouts. Any aggregate remainder consequently has neither a claimant nor a release path: liquidity dust remains reserved, and partial-redemption DS-token dust remains in the vault. See contracts/AsyncFundVault.sol:679-687 and contracts/AsyncFundVault.sol:899-955.

Impact: Normal multi-controller redemptions can permanently leave small liquidity balances unavailable to the reserve and unfulfilled DS-token balances unavailable to their controllers. These residual amounts can accumulate across settled generations.

Recommended Mitigation: Track each fulfilled generation's remaining liquidity and unfulfilled shares. Decrement those values as claims are processed and allocate each final residual deterministically to the last claimant, or explicitly release a fully settled residual under a documented distribution rule.

Securitize: Acknowledged; we're accepting this as-is with no code change, for a few reasons:

- Directionally safe. The dust is unclaimed value sitting in the vault's custody, not a shortfall owed to anyone; it favors the protocol/remaining investors rather than creating any loss.
- Magnitude is immaterial relative to fund operations; bounded to at most (claimants - 1) base units per partial-fill generation, and partial fills themselves are the exception, not the norm.
- Recovery already exists at the right layer. The liquidity-side residual isn't actually stranded; `totalClaimableRedemptionLiquidity` only tracks the true remaining obligation, so once claims settle, the residual becomes ordinary `reserveBalance` headroom recoverable through the existing `withdrawReserve` path. The DS-Token-side residual can be recovered directly by the Issuer/Transfer Agent via the DS Token's own `seize/burn` functions (`onlyTransferAgentOrAbove` / `onlyIssuerOrTransferAgentOrAbove`) which operate on any holder, including the vault, with no vault-side involvement required.

Given that, we don't think an ERC-7540-layer admin function is warranted here; it'd duplicate a capability the token layer already provides, for a problem whose value doesn't justify the added surface area.

7.3.4 `AsyncFundVault::deposit` strands zero-share fulfilled deposits

Description: `AsyncFundVault::deposit` rejects a zero `claimedShares` value before clearing the fulfilled request; `mint` has the same condition. Because `cancelDepositRequest` only accepts an Active generation, the fulfilled assets cannot be refunded through cancellation. This state is reachable when the DS token has materially fewer decimals than the liquidity token, or when the locked NAV is high enough that the fulfilled asset amount rounds below one DS-token base unit. See `contracts/AsyncFundVault.sol:481-493`, `contracts/AsyncFundVault.sol:515-523`, and `contracts/AsyncFundVault.sol:532-559`.

Impact: The zero-share fulfilled request has no standalone refund or claim path. If another fulfilled generation later makes the controller's aggregate claim produce nonzero shares, the clear loop consumes the zero-share request as part of the aggregate claim without minting value for that request.

Recommended Mitigation: When a fulfilled deposit conversion produces zero shares, clear the request, reduce `reserveBalance`, and return the recorded assets to the controller. Reserve-withdrawal accounting must preserve the refundable amount until the cleanup executes; otherwise a manager can withdraw the custody needed for the refund after fulfillment. Alternatively, enforce a request minimum that is safe under the settlement policy while retaining the refund route when the locked NAV still produces zero shares.

Securitize: Fixed in commits [65a7b33](#), [2208c63](#).

Cyfrin: Verified.

7.3.5 `AsyncFundVault::fulfillRedemptions` burns zero-value redemptions

Description: For a nonzero pending redemption whose conversion rounds down to zero liquidity, `AsyncFundVault::fulfillRedemptions` enters the branch shared with an empty generation. It sets the fulfillment rate to `WAD`, commits no liquidity, and then burns all pending shares. The fulfilled request can subsequently be cleared without any liquidity payout. See `contracts/AsyncFundVault.sol:649-687`.

Impact: A valid nonzero redemption that is below the liquidity token's settlement precision loses its escrowed shares while receiving no liquidity.

Recommended Mitigation: Handle a nonzero `totalShares` with zero `totalRedemptionValue` as an unfulfilled redemption: set the fulfillment rate to zero and do not burn the escrowed shares. Alternatively, enforce a redemption minimum guaranteed to produce at least one liquidity-token base unit at settlement.

Securitize: Fixed in commit [d7e314f](#).

Cyfrin: Verified.

7.3.6 AsyncFundVault::withdraw rejects zero-fill redemption claims

Description: A fulfilled redemption with no committed liquidity still has positive claimable shares and must return its unfulfilled DS tokens. AsyncFundVault::withdraw instead treats totalLiquidity == 0 as no claim and reverts, whereas redeem checks totalShares and can execute the same zero-liquidity claim. See contracts/AsyncFundVault.sol:708-752 and contracts/AsyncFundVault.sol:899-955.

Impact: Users and integrations using withdraw cannot finalize a zero-fill redemption through that supported claim path. They must switch to redeem to recover the escrowed unfulfilled shares.

Recommended Mitigation: In AsyncFundVault::withdraw, use the same totalShares claim-existence check as redeem, then retain the exact-assets equality check so a zero-liquidity claim can execute and return unfulfilled shares.

Securitize: Fixed in commit [2e2bf00](#).

Cyfrin: Verified.

7.3.7 ERC 7540 and ERC 7575 conformance gaps break standard discovery and claim availability views

Description: The vault advertises conformance with ERC-7540, but its request identifiers, interface discovery, share token lookup, and maximum claim views do not implement several requirements of the final standard. Generic ERC-7540 routers and indexers can consequently reject the vault, misclassify requests, or conclude that a valid claim is unavailable.

The first deposit and redemption generations both receive identifier zero because each counter starts at zero and is incremented after assignment. Later generations return nonzero identifiers. ERC-7540 specifies that once a vault returns zero for any request identifier, it must return zero for every request. The vault instead changes identifier models after its first generation.

```
// contracts/AsyncFundVault.sol:270-272
generationId = $.depositGenerationCount++;
$.depositGenerations[generationId].status = GenerationStatus.Active;
$.currentDepositGenerationId = generationId;

// contracts/AsyncFundVault.sol:327-329
generationId = $.redeemGenerationCount++;
$.redeemGenerations[generationId].status = GenerationStatus.Active;
$.currentRedeemGenerationId = generationId;
```

The final ERC-7540 specification also requires ERC-165 discovery for the operator, deposit request, redemption request, and ERC-7575 interfaces. It requires ERC-7575 support, in particular a nonreverting IERC7575::share function that returns the external DS Token address. AsyncFundVault does not define IERC7575::share, and its inherited AsyncFundVault::supportsInterface implementation does not return the required identifiers. The normative requirements are documented at <https://eips.ethereum.org/EIPS/eip-7540> and <https://eips.ethereum.org/EIPS/eip-7575>.

Finally, the maximum functions describe request intake or return fixed zero values instead of exposing the caller's claimable state. ERC-7540 uses the existing ERC-4626 functions as claim entry points and states that IERC4626::maxDeposit increases and decreases with IERC7540::claimableDepositRequest. AsyncFundVault::maxDeposit returns type(uint256).max whenever subscriptions are open, even when the queried controller has no claimable deposit, while AsyncFundVault::maxWithdraw and AsyncFundVault::maxRedeem return zero even when the controller has a fulfilled redemption.

```
// contracts/AsyncFundVault.sol:192-212
function maxDeposit(address /* receiver */ ) external view returns (uint256) {
    VaultStorage storage $ = _getStorage();
    return ($.subscriptionsEnabled && !paused()) ? type(uint256).max : 0;
}

function maxMint(address /* receiver */ ) external view returns (uint256) {
    VaultStorage storage $ = _getStorage();
```

```

    return ($.subscriptionsEnabled && !paused()) ? type(uint256).max : 0;
}

function maxWithdraw(address /* owner */ ) external pure returns (uint256) {
    return 0;
}

function maxRedeem(address /* owner */ ) external pure returns (uint256) {
    return 0;
}

```

Impact: Standards-aware integrations may fail to discover the asynchronous interfaces, resolve the DS Token through `share()`, or determine claim availability from the `max*` functions. Mixed request-ID semantics can also cause indexers to merge or mislabel requests; protocol-specific users can still transact, so the primary impact is failed or omitted integration activity rather than direct asset loss.

Recommended Mitigation: Use one ERC-7540 request-ID model consistently, implement ERC-7575 `share()`, and report all required interface IDs through `supportsInterface`. Make `maxDeposit`, `maxMint`, `maxWithdraw`, and `maxRedeem` reflect each controller's currently claimable amounts.

Securitize: Fixed in commit [20461a7](#).

Cyfrin: Verified.

7.3.8 Aggregated deposit claims can exceed DS Token compliance limits

Description: `AsyncFundVault::requestDeposit` does not check whether the resulting DS Tokens can be issued to the receiver. After fulfillment, `AsyncFundVault::deposit` and `AsyncFundVault::mint` aggregate all claimable generations and issue the full amount in one call:

```
$.dsToken.issueTokens(receiver, claimedShares);
```

DS Token compliance can reject issuance when:

```
receiver balance + claimed shares > maximum holdings
```

This can occur because multiple claims are aggregated or because a positive rebase increases the receiver's balance after the request. The vault does not support partial claims, and fulfilled deposits cannot be cancelled.

Impact: The claim may remain blocked until the receiver reduces their balance, another compliant receiver is selected, or an administrator intervenes. If the aggregate amount exceeds the limit for every ordinary receiver, it cannot be claimed through the normal flow.

Recommended Mitigation: Support partial claims or split fulfilled entitlements into compliant amounts. Provide a refund or administrative recovery path for fulfilled deposits that cannot pass issuance compliance.

Securitize: Acknowledged; this fund does not configure maximum or minimum holdings limits.

7.3.9 `AsyncFundVault::totalAssets` reports a mutable liquidity reserve rather than total managed assets

Description: `AsyncFundVault::totalAssets` returns `reserveBalance`, although this value represents only liquidity currently held by the contract. `MANAGER_ROLE` can increase or decrease it using `AsyncFundVaultAdmin::injectLiquidity` and `AsyncFundVaultAdmin::withdrawReserve` without any corresponding DS Token issuance or burn.

It also includes cancellable deposits and liquidity committed to redemption claims. Therefore, `reserveBalance` represents neither net available liquidity nor the total assets economically backing outstanding DS Tokens issued by vault.

Impact: ERC-4626 integrations cannot reliably use `AsyncFundVault::totalAssets` because it reports only the vault's mutable liquidity reserve, not the total assets backing the DS Token.

Recommended Mitigation: Document that the contract is not compatible with integrations relying on ERC-4626 `totalAssets` accounting.

Securitize: Fixed in commit [c315a6d](#).

Cyfrin: Verified.

7.3.10 Zero live NAV bypasses settlement price validation

Description: `AsyncFundVault::_checkNavPrice` skips validation when the normalized live NAV is zero:

```
uint256 live =
    $.navProvider.rate() * WAD
    / (10 ** uint256($.dsDecimals));

if (live == 0) return;
```

The settler can consequently fulfill deposits or redemptions using any nonzero `navPriceWAD`, bypassing the configured tolerance.

Impact: An incorrect settlement price can cause depositors to receive the wrong number of DS Tokens or redeemers to receive the wrong liquidity amount, resulting in irreversible accounting losses.

Recommended Mitigation: Revert when the live NAV is zero:

```
if (live == 0) revert InvalidLiveNavPrice();
```

Settlement should resume only after the NAV provider returns a valid price.

Securitize: Fixed in commit [5ad7619](#).

Cyfrin: Verified.

7.3.11 Cancellation functions do not support an alternative recipient

Description: Both cancellation functions always return tokens directly to the controller:

```
// Deposit cancellation
$.liquidityToken.safeTransfer(controller, amount);

// Redemption cancellation
IERC20(address($.dsToken)).safeTransfer(controller, amount);
```

Neither function allows the caller to specify another recipient, unlike claim functions such as `deposit()` and `redeem()`.

If the controller is blocked from receiving the liquidity token or DS Token, the corresponding cancellation reverts. The user can wait for fulfillment, but cannot redirect the cancelled tokens to another valid address.

Recommended Mitigation: Add a recipient parameter to both cancellation functions.

Securitize: Fixed in commit [a97c952](#).

Cyfrin: Verified.

7.4 Informational

7.4.1 AsyncFundVault::redeem permits vault-self claim receivers

Description: The claim entry points reject only the zero address, so an authorized controller can select the vault itself as receiver in deposit or mint (contracts/AsyncFundVault.sol:471-523) and in redeem or withdraw (contracts/AsyncFundVault.sol:708-751). Deposit claims are cleared before DS tokens are issued to that receiver. Redemption claims similarly clear the per-generation state and reduce both reserveBalance and totalClaimableRedemptionLiquidity before transferring liquidity to the supplied receiver (contracts/AsyncFundVault.sol:906-943). A transfer or issuance to the vault itself leaves the value at the vault while the corresponding claim and accounting obligations have already been cleared.

Impact: An authorized controller or approved operator can irreversibly consume the controller's fulfilled claim without delivering usable value to a user-controlled receiver. For redemption claims, the real liquidity balance remains in the vault but the tracked reserve decreases, so later operations may calculate less available liquidity than the vault actually holds. For deposit claims, DS tokens issued to the vault are stranded outside a user's usable redemption flow.

Recommended Mitigation: Add a shared receiver validation helper that rejects both the zero address and address(this), then call it in deposit, mint, redeem, and withdraw before any claim state is cleared or value is transferred.

Securitize: Fixed in commit [629d543](#).

Cyfrin: Verified.

7.4.2 AsyncFundVault::initialize accepts non-contract NAV providers

Description: AsyncFundVault::initialize forwards navProvider_ to the base initializer at contracts/AsyncFundVault.sol:104-109. That initializer rejects only the zero address before storing the provider at contracts/base/AsyncFundVaultAdmin.sol:109-128, so a nonzero externally owned account can be configured. _checkNavPrice subsequently invokes navProvider.rate() at contracts/AsyncFundVault.sol:960-969; the call to a codeless address returns empty returndata, and the ABI decoder reverts on the missing uint256 return value before live can be assigned or the live == 0 skip can be reached. Both settlement flows call this check before changing generation state at contracts/AsyncFundVault.sol:438-445 and contracts/AsyncFundVault.sol:633-640, so the invalid configuration prevents deposit and redemption generations from being fulfilled.

Impact: A deployment-time provider-address error leaves submitted requests unfulfillable until an administrator deploys and executes a recovery upgrade that replaces the provider.

Recommended Mitigation: In AsyncFundVaultAdmin::_AsyncFundVaultAdmin_init, reject a navProvider_ whose code.length is zero before assigning it. Also validate the dependency's expected response during initialization by requiring a successful rate call with a well-formed return value.

Securitize: Acknowledged.

7.4.3 Self reassignment temporarily hides claims from aggregate indexes until administrator repair

Description: Calling either claimable-balance reassignment function with the same source and destination preserves the controller's per-generation balance while removing the generation from the index used by every aggregate claim path. The affected controller cannot claim the deposit or redemption through the aggregate paths until an administrator repairs the index, and a redemption leaves its committed liquidity locked during that interval.

AsyncFundVaultAdmin::reassignClaimableDeposit does not reject from == to. When both arguments identify the same controller, toHadNoPriorClaim is false because the controller already has a nonzero balance. The function then clears and restores the same mapping slot, removes the generation from controllerDepositGenerations, and does not add it back. AsyncFundVaultAdmin::reassignClaimableRedemption repeats the same sequence for redemption claims.

```
// contracts/base/AsyncFundVaultAdmin.sol:242-266
function reassignClaimableDeposit(uint256 generationId, address from, address to)
```



```

    external
    onlyRole(DEFAULT_ADMIN_ROLE)
{
    if (to == address(0)) revert ZeroAddress();

    VaultStorage storage $ = _getStorage();

    if ($.depositGenerations[generationId].status != GenerationStatus.Fulfilled) {
        revert GenerationNotFulfilled(generationId);
    }

    uint256 amount = $.pendingDepositAssets[generationId][from];
    if (amount == 0) revert NoRequestInGeneration(generationId, from);

    bool toHadNoPriorClaim = $.pendingDepositAssets[generationId][to] == 0;
    $.pendingDepositAssets[generationId][from] = 0;
    $.pendingDepositAssets[generationId][to] += amount;

    _removeFromList($.controllerDepositGenerations[from], generationId);
    if (toHadNoPriorClaim) {
        $.controllerDepositGenerations[to].push(generationId);
    }
}

// contracts/base/AsyncFundVaultAdmin.sol:272-294
function reassignClaimableRedemption(uint256 generationId, address from, address to)
    external
    onlyRole(DEFAULT_ADMIN_ROLE)
{
    if (to == address(0)) revert ZeroAddress();

    VaultStorage storage $ = _getStorage();

    if ($.redeemGenerations[generationId].status != GenerationStatus.Fulfilled) {
        revert GenerationNotFulfilled(generationId);
    }

    uint256 shares = $.pendingRedeemShares[generationId][from];
    if (shares == 0) revert NoRequestInGeneration(generationId, from);

    bool toHadNoPriorClaim = $.pendingRedeemShares[generationId][to] == 0;
    $.pendingRedeemShares[generationId][from] = 0;
    $.pendingRedeemShares[generationId][to] += shares;

    _removeFromList($.controllerRedeemGenerations[from], generationId);
    if (toHadNoPriorClaim) {
        $.controllerRedeemGenerations[to].push(generationId);
    }
}

```

The direct request views continue to read the nonzero mapping entries. In contrast, `AsyncFundVault::_computeClaimableDepositTotals` and `AsyncFundVault::_computeClaimableRedemptionTotals` enumerate only the corresponding controller generation arrays. Once self-reassignment removes the generation, `AsyncFundVault::deposit`, `AsyncFundVault::mint`, `AsyncFundVault::redeem`, and `AsyncFundVault::withdraw` all observe zero aggregate claimable value and revert before clearing the request.

```

// contracts/AsyncFundVault.sol:832-848
uint256[] storage genIds = $.controllerDepositGenerations[controller];
uint256 len = genIds.length;
for (uint256 i; i < len; ++i) {
    uint256 genId = genIds[i];
    uint256 amount = $.pendingDepositAssets[genId][controller];
}

```

```

    if (amount > 0 && $.depositGenerations[genId].status == GenerationStatus.Fulfilled) {
        claimedAssets += amount;
        claimedShares += _assetsToShares(
            amount,
            $.depositGenerations[genId].navPrice,
            $.dsDecimals,
            $.liquidityDecimals
        );
    }
}

// contracts/AsyncFundVault.sol:875-892
uint256[] storage genIds = $.controllerRedeemGenerations[controller];
uint256 len = genIds.length;
for (uint256 i; i < len; ++i) {
    uint256 genId = genIds[i];
    uint256 amount = $.pendingRedeemShares[genId][controller];
    if (amount > 0 && $.redeemGenerations[genId].status == GenerationStatus.Fulfilled) {
        RedemptionGenerationData storage gen = $.redeemGenerations[genId];
        totalShares += amount;
        totalLiquidity += (amount * gen.totalLiquidity) / gen.totalPendingShares;
    }
}

```

Impact: Self-reassignment can hide an otherwise valid deposit or redemption claim from aggregate lookup and continue reserving associated liquidity. Only `DEFAULT_ADMIN_ROLE` can trigger and repair the inconsistency, so it is an operationally induced temporary denial of claim access rather than an unprivileged exploit.

Recommended Mitigation: Reject `from == to` with a dedicated error in both reassignment functions before mutating balances or indexes, preserving the mapping-to-index invariant and surfacing operator mistakes.

Securitize: Fixed in commit [7999a4f](#).

Cyfrin: Verified.

7.4.4 Partial redemption claims require administrative recovery if the controller loses DS Token eligibility

Description: When a redemption generation is only partially fulfilled, `AsyncFundVault::redeem` and `AsyncFundVault::withdraw` perform two transfers atomically:

1. Transfer the fulfilled liquidity amount to receiver.
2. Return the unfulfilled DS Tokens to controller.

```

if (totalLiquidity > 0) {
    $.liquidityToken.safeTransfer(receiver, totalLiquidity);
}

if (totalUnfulfilled > 0) {
    IERC20(address($.dsToken)).safeTransfer(
        controller,
        totalUnfulfilled
    );
}

```

The DS Token applies compliance checks to transfers. If the controller loses eligibility after requesting redemption but before claiming—for example, because of expired KYC, blacklisting, or wallet restrictions—the transfer of unfulfilled DS Tokens reverts.

Impact: A controller who loses DS Token eligibility cannot claim:

- The liquidity corresponding to the fulfilled part of the redemption.
- The DS Tokens corresponding to the unfulfilled part.

Recovery requires restoring the controller's compliance status or intervention from `DEFAULT_ADMIN_ROLE` through `AsyncFundVaultAdmin::reassignClaimableRedemption` to move the claim to an eligible controller.

Recommended Mitigation: Document the required recovery procedure for controllers that lose DS Token eligibility. Another option is to send DS tokens to the receiver instead of controller.

Securitize: Fixed in commit [a97c952](#).

Cyfrin: Verified.

7.5 Gas Optimization

7.5.1 Cache request balances that are checked before an additive update

Description: The request and reassignment flows read a pending-balance mapping entry to determine whether to add a generation to a controller list, then read the same entry again for the additive update. Cache the initial value and perform a single calculated write to avoid the repeat warm mapping read on the successful path.

```
contracts/AsyncFundVault.sol
389:     if ($.pendingDepositAssets[genId][controller] == 0) {
395:         $.pendingDepositAssets[genId][controller] += assets;
585:     if ($.pendingRedeemShares[genId][controller] == 0) {
591:         $.pendingRedeemShares[genId][controller] += shares;

contracts/base/AsyncFundVaultAdmin.sol
258:     bool toHadNoPriorClaim = $.pendingDepositAssets[generationId][to] == 0;
260:     $.pendingDepositAssets[generationId][to] += amount;
287:     bool toHadNoPriorClaim = $.pendingRedeemShares[generationId][to] == 0;
289:     $.pendingRedeemShares[generationId][to] += shares;
```

Recommended Mitigation: In `contracts/AsyncFundVault.sol`, load each pending balance into a local before the zero check, use that value for the list decision, and assign the computed sum rather than using `+=`:

```
uint256 pendingAssets = $.pendingDepositAssets[genId][controller];
if (pendingAssets == 0) {
    if ($.controllerDepositGenerations[controller].length >= MAX_OUTSTANDING_GENERATIONS) {
        revert TooManyOutstandingGenerations();
    }
    $.controllerDepositGenerations[controller].push(genId);
}
$.pendingDepositAssets[genId][controller] = pendingAssets + assets;
```

Apply the same cache-and-single-write pattern to the redemption request site. For `reassignClaimableDeposit` and `reassignClaimableRedemption`, preserve the current same-address semantics: because the source slot is zeroed before the destination is incremented, any destination caching must re-read after the zeroing step or apply only when `from != to`.

Securitize: Fixed in commit [7574b36](#).

Cyfrin: Verified.

7.5.2 Use the already-snapshotted list length while compacting claim lists

Description: Both claim-compaction helpers cache the original list length for their scan, but their subsequent pop loops re-read `genIds.length` from storage for every iteration. Count down from the cached length, or from `len - writeIdx`, to avoid the redundant warm storage read for each removed generation.

```
contracts/AsyncFundVault.sol
855:     uint256[] storage genIds = $.controllerDepositGenerations[controller];
856:     uint256 len = genIds.length;
870:     while (genIds.length > writeIdx) genIds.pop();
913:     uint256[] storage genIds = $.controllerRedeemGenerations[controller];
914:     uint256 len = genIds.length;
933:     while (genIds.length > writeIdx) genIds.pop();
```

Recommended Mitigation: In both compaction helpers in `contracts/AsyncFundVault.sol`, use a memory counter for the number of entries to remove. `pop` still performs its required storage update, but the loop predicate no longer needs an additional storage read:

```
for (uint256 remaining = len - writeIdx; remaining != 0; --remaining) {
    genIds.pop();
}
```

Securitize: Acknowledged.

7.5.3 Pack the tolerance with the cached token decimal fields

Description: VaultStorage leaves the two cached decimal fields in a mostly empty slot, then places the two-byte navPriceTolerance after the operators mapping, forcing it into a new slot. Moving the tolerance immediately after the decimal fields makes all three values share a slot. The layout uses slots 0 through 17 before the move and slots 0 through 16 after it, reducing the layout from 18 to 17 slots. This saves one initialized storage slot per newly deployed vault and can make the tolerance read warm after the decimal reads in NAV checks.

```
contracts/base/AsyncFundVaultStorage.sol
154:      // Token metadata cache
155:      /// @dev DS Token decimal count, cached from `dsToken.decimals()` at initialisation.
156:      uint8 dsDecimals;
157:      /// @dev Liquidity token decimal count, cached from `liquidityToken.decimals()` at
158:      ///      initialisation. Required for WAD navPrice conversion arithmetic.
159:      uint8 liquidityDecimals;
161:      // Operators
162:      /// @dev operators[controller][operator] = approved.
163:      mapping(address controller => mapping(address operator => bool approved)) operators;
165:      // NAV price tolerance
169:      uint16 navPriceTolerance;
```

Recommended Mitigation: Move navPriceTolerance directly after liquidityDecimals in contracts/base/AsyncFundVaultStorage.sol so all three values occupy the same slot:

```
uint8 dsDecimals;
uint8 liquidityDecimals;
uint16 navPriceTolerance;
mapping(address controller => mapping(address operator => bool approved)) operators;
```

Securitize: Fixed in commit [bc377f3](#).

Cyfrin: Verified.

7.5.4 Cache each reused generation mapping entry as a storage reference

Description: The request, cancellation, and claim paths access multiple fields of the same generation mapping entry but repeatedly index depositGenerations[genId] or redeemGenerations[genId]. Bind the entry once as a storage reference and access its fields through that reference to avoid recomputing the nested mapping location. Verify the generated bytecode for the configured compilation pipeline because this reuse is not reliably preserved across branch boundaries by every pipeline.

```
contracts/AsyncFundVault.sol
383:      if ($.depositGenerations[genId].status != GenerationStatus.Active) {
396:          $.depositGenerations[genId].totalPendingDeposits += assets;
542:          GenerationStatus status = $.depositGenerations[generationId].status;
552:          $.depositGenerations[generationId].totalPendingDeposits -= amount;
580:      if ($.redeemGenerations[genId].status != GenerationStatus.Active) {
592:          $.redeemGenerations[genId].totalPendingShares += shares;
767:          GenerationStatus status = $.redeemGenerations[generationId].status;
777:          $.redeemGenerations[generationId].totalPendingShares -= amount;
844:          if (amount > 0 && $.depositGenerations[genId].status == GenerationStatus.Fulfilled) {
846:              claimedShares += _assetsToShares(amount, $.depositGenerations[genId].navPrice,
847:          ↪ $.dsDecimals, $.liquidityDecimals);
887:          if (amount > 0 && $.redeemGenerations[genId].status == GenerationStatus.Fulfilled) {
888:              RedemptionGenerationData storage gen = $.redeemGenerations[genId];
921:          if (amount > 0 && $.redeemGenerations[genId].status == GenerationStatus.Fulfilled) {
922:              RedemptionGenerationData storage gen = $.redeemGenerations[genId];
```

Recommended Mitigation: In `contracts/AsyncFundVault.sol`, resolve the generation entry before its first field access and reuse it for the status check and subsequent field updates:

```
DepositGenerationData storage gen = $.depositGenerations[genId];  
if (gen.status != GenerationStatus.Active) revert NoActiveGeneration();  
// ...  
gen.totalPendingDeposits += assets;
```

Use the corresponding `RedemptionGenerationData` storage `gen` form at the redemption and claim-loop locations.

Securitize: Acknowledged.